



Computer Networks

CS3611

Application Layer-Part 1

Haiming Jin

The slides are adapted from those provided by Prof. Romit Roy Choudhury.

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP, POP3, IMAP

Chapter 2: Application Layer

Our goals:

- ❑ Principles of network application design
 - ❖ transport-layer service models
 - ❖ client-server paradigm
 - ❖ peer-to-peer paradigm

- ❑ Popular protocols through case studies
 - ❖ HTTP
 - ❖ FTP
 - ❖ SMTP / POP3 / IMAP
 - ❖ DNS
- ❑ Network programming
 - ❖ socket API

Some network apps

- ☐ E-mail
- ☐ Web
- ☐ Instant messaging
- ☐ Remote login
- ☐ P2P file sharing
- ☐ Multi-user network games
- ☐ Streaming stored video clips
- ☐ Internet telephone
- ☐ Real-time video conference
- ☐ Massive parallel computing
- ☐
- ☐
- ☐

Next generation: The network will be the computer. Most Applications will run over the network. Local PC minimaly required
Example: Google spread sheet

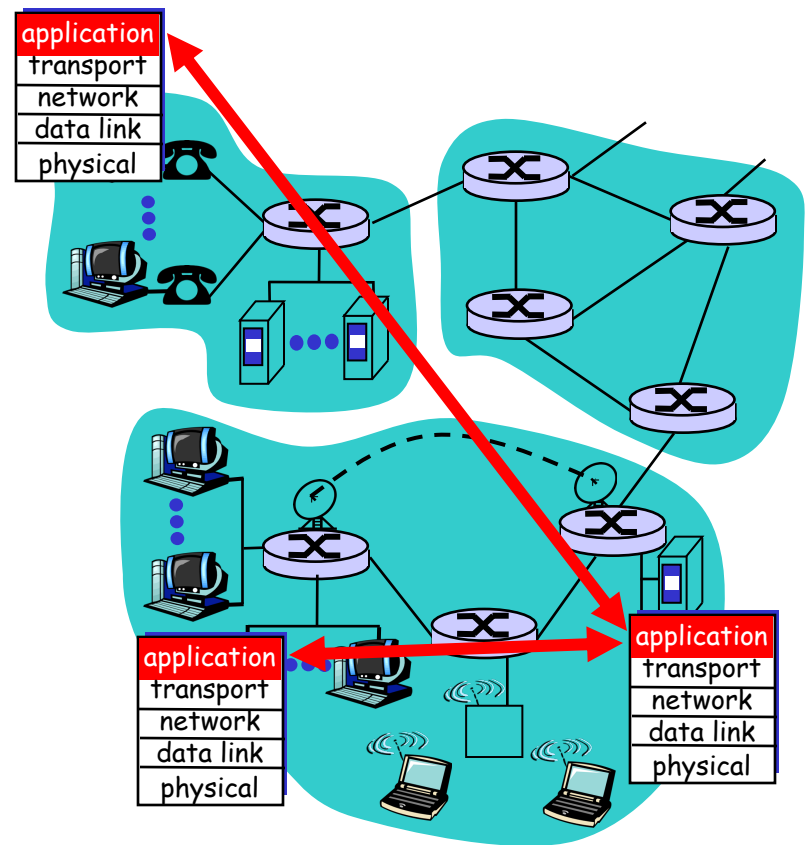
Creating a network app

Write programs that

- ❖ run on different end systems and
- ❖ communicate over a network.
- ❖ e.g., Web: Web server software communicates with browser software

little software written for devices in network core

- ❖ network core devices do not run user application code
- ❖ application on end systems allows for rapid app development, propagation



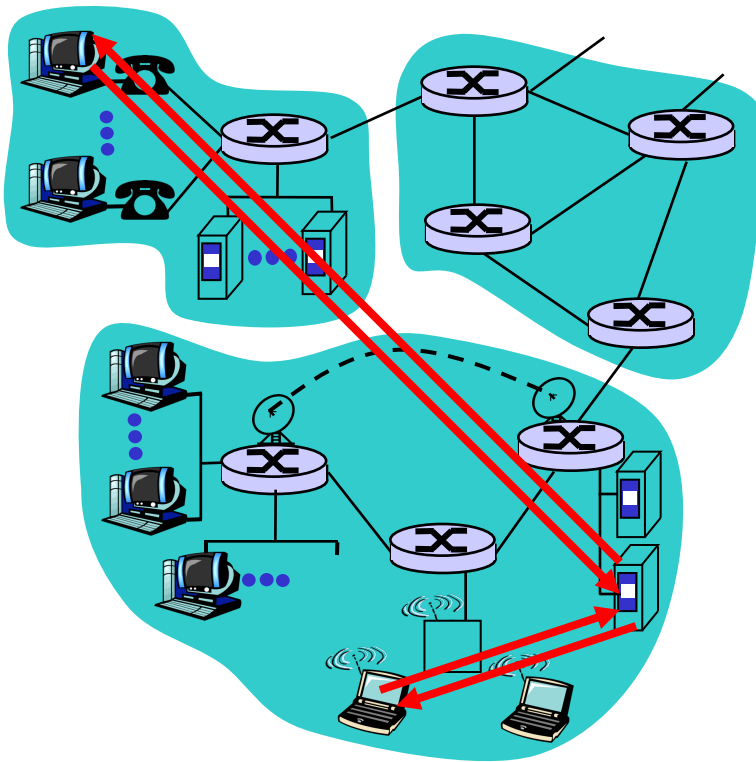
Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS

Application architectures

- ❑ Client-server
- ❑ Peer-to-peer (P2P)
- ❑ Hybrid of client-server and P2P

Client-server architecture



server:

- ❖ always-on host
- ❖ permanent IP address
- ❖ server farms for scaling

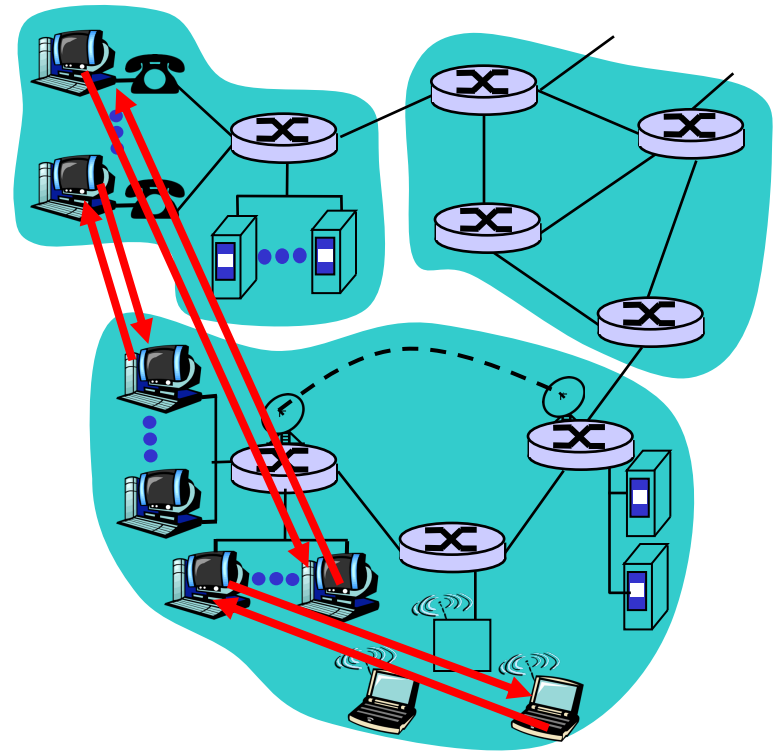
clients:

- ❖ communicate with server
- ❖ may be intermittently connected
- ❖ may have dynamic IP addresses
- ❖ do not communicate directly with each other

Pure P2P architecture

- ❑ no always-on server
- ❑ arbitrary end systems directly communicate
- ❑ peers are intermittently connected and change IP addresses
- ❑ example: Gnutella

Highly scalable but difficult to manage



Hybrid of client-server and P2P

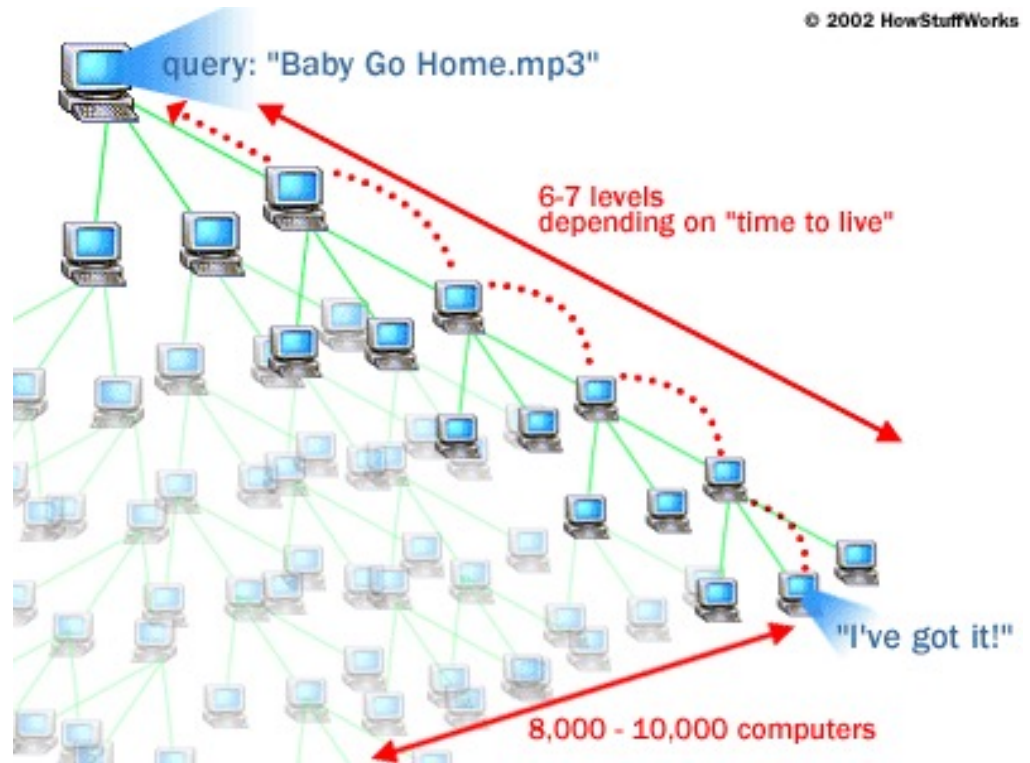
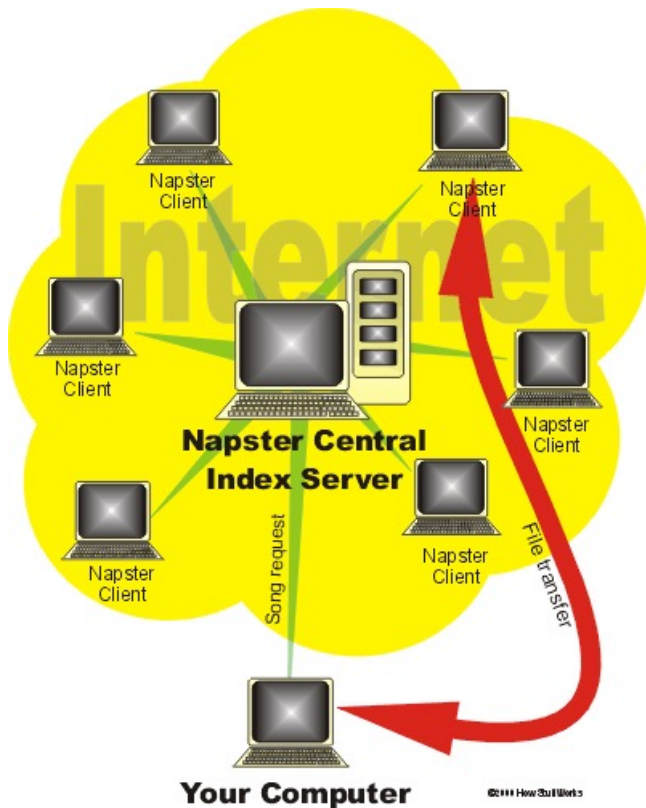
Skype

- ❖ Internet telephony app
- ❖ Finding address of remote party: centralized server(s)
- ❖ Client-client connection is direct (not through server)

Instant messaging

- ❖ Chatting between two users is P2P
- ❖ Presence detection/location centralized:
 - User registers its IP address with central server when it comes online
 - User contacts central server to find IP addresses of buddies

Case Study: Napster Vs Gnutella



Any problem with this architecture?

Processes communicating

Process: program running within a host.

- within same host, two processes communicate using **inter-process communication** (defined by OS).
- processes in different hosts communicate by exchanging **messages**

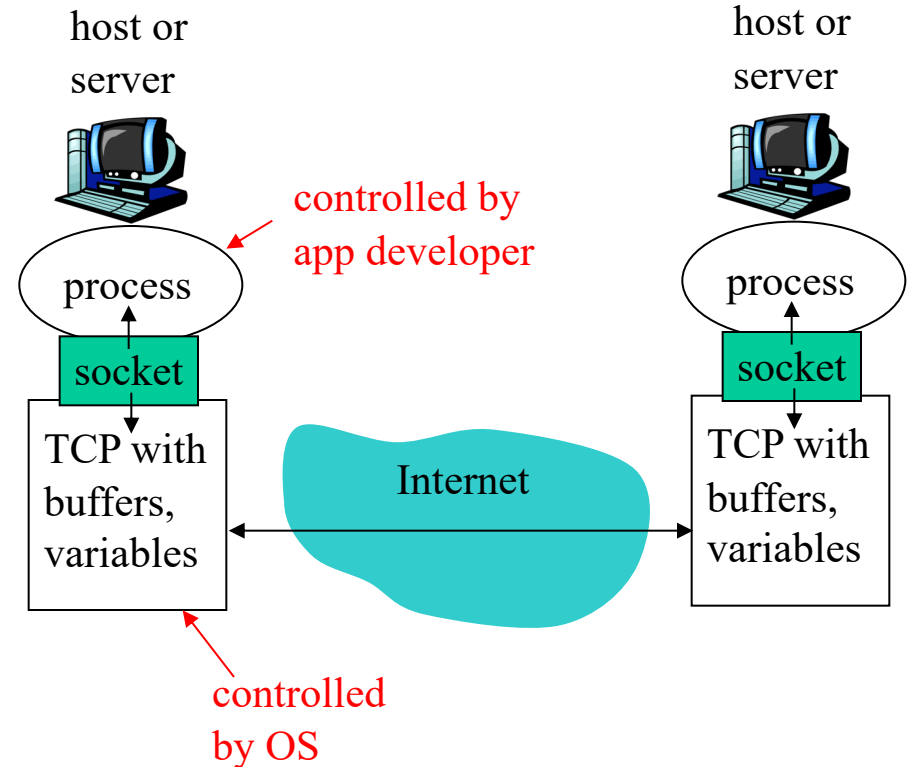
Client process: process that initiates communication

Server process: process that waits to be contacted

- Note: applications with P2P architectures have client processes & server processes

Sockets

- ❑ process sends/receives messages to/from its **socket**
- ❑ socket analogous to door
 - ❖ sending process shoves message out door
 - ❖ sending process relies on transport infrastructure on other side of door which brings message to socket at receiving process



- ❑ API: (1) choice of transport protocol; (2) ability to fix a few parameters (lots more on this later)

Addressing processes

- ❑ to receive messages,
process must have
identifier
- ❑ host device has unique 32-bit IP address
- ❑ **Q:** does IP address of host
on which process runs
suffice for identifying the
process?

Addressing processes

- ❑ to receive messages, process must have *identifier*
- ❑ host device has unique 32-bit IP address
- ❑ **Q:** does IP address of host on which process runs suffice for identifying the process?
 - ❖ **Answer:** NO, many processes can be running on same host
- ❑ *identifier* includes both **IP address** and **port numbers** associated with process on host.
- ❑ Example port numbers:
 - ❖ HTTP server: 80
 - ❖ Mail server: 25
- ❑ to send HTTP message to gaia.cs.umass.edu web server:
 - ❖ IP address: 128.119.245.12
 - ❖ Port number: 80
- ❑ more shortly...

Message Format:

App-layer protocol defines

- ❑ Types of messages exchanged,
 - ❖ e.g., request, response
- ❑ Message syntax:
 - ❖ what fields in messages & how fields are delineated
- ❑ Message semantics
 - ❖ meaning of information in fields
- ❑ Rules for when and how processes send & respond to messages

Public-domain protocols:

- ❑ defined in RFCs
- ❑ allows for interoperability
- ❑ e.g., HTTP, SMTP

Proprietary protocols:

- ❑ e.g., Skype

Requirements for Message Transport:

Data loss

- ❑ some apps (e.g., audio) can tolerate some loss
- ❑ other apps (e.g., file transfer, telnet) require 100% reliable data transfer

Timing

- ❑ some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

Bandwidth

- ❑ some apps (e.g., multimedia) require minimum amount of bandwidth to be “effective”
- ❑ other apps (“elastic apps”) make use of whatever bandwidth they get

Why is **bandwidth** different from **timing** constraints?

Internet transport protocols services

TCP service:

- ❑ *connection-oriented*: setup required between client and server processes
- ❑ *reliable transport* between sending and receiving process
- ❑ *flow control*: sender won't overwhelm receiver
- ❑ *congestion control*: throttle sender when network overloaded
- ❑ *does not provide*: timing, minimum bandwidth guarantees

UDP service:

- ❑ unreliable data transfer between sending and receiving process
- ❑ does not provide: connection setup, reliability, flow control, congestion control, timing, or bandwidth guarantee

Q: why bother? Why is there a UDP?

Internet apps: application, transport protocols

	Application	Application layer protocol	Underlying transport protocol
	e-mail	SMTP [RFC 2821]	TCP
remote	terminal access	Telnet [RFC 854]	TCP
	Web	HTTP [RFC 2616]	TCP
	file transfer	FTP [RFC 959]	TCP
streaming	multimedia	HTTP (e.g. Youtube)	TCP or UDP
	Internet telephony	proprietary (e.g., Skype)	typically UDP

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
 - ❖ app architectures
 - ❖ app requirements
- ❑ 2.2 Web and HTTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 P2P file sharing
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP
- ❑ 2.9 Building a Web server

Web and HTTP

First some jargon

- ❑ Web page consists of objects
- ❑ Object can be HTML file, JPEG image, Java applet, audio file,...
- ❑ Web page consists of base HTML-file which includes several referenced objects
- ❑ Each object is addressable by a URL
- ❑ Example URL:

`www.someschool.edu/someDept/pic.gif`

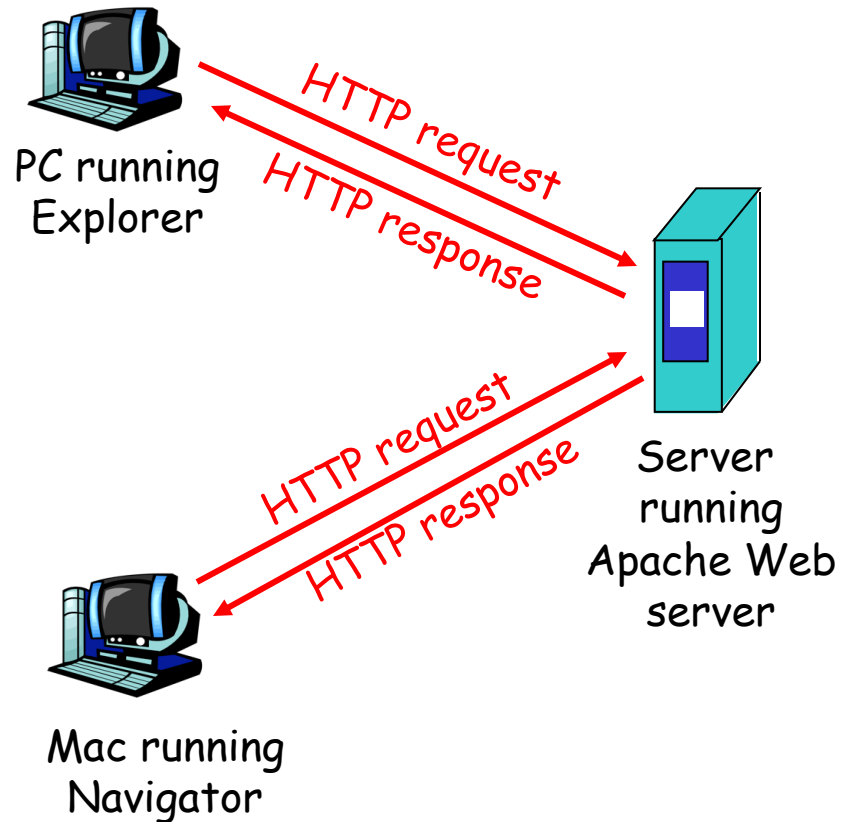
host name

path name

HTTP overview

HTTP: hypertext transfer protocol

- ❑ Web's application layer protocol
- ❑ client/server model
 - ❖ *client*: browser that requests, receives, “displays” Web objects
 - ❖ *server*: Web server sends objects in response to requests
- ❑ HTTP 1.0: RFC 1945
- ❑ HTTP 1.1: RFC 2068



HTTP overview (continued)

Uses TCP:

- ❑ client initiates TCP connection (creates socket) to server, port 80
- ❑ server accepts TCP connection from client
- ❑ HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- ❑ TCP connection closed

HTTP is “stateless”

- ❑ server maintains no information about past client requests

aside
Protocols that maintain “state” are complex!

- ❑ past history (state) must be maintained
- ❑ if server/client crashes, their views of “state” may be inconsistent, must be reconciled

HTTP connections

Nonpersistent HTTP

- ❑ At most one object is sent over a TCP connection.
- ❑ HTTP/1.0 uses nonpersistent HTTP

Persistent HTTP

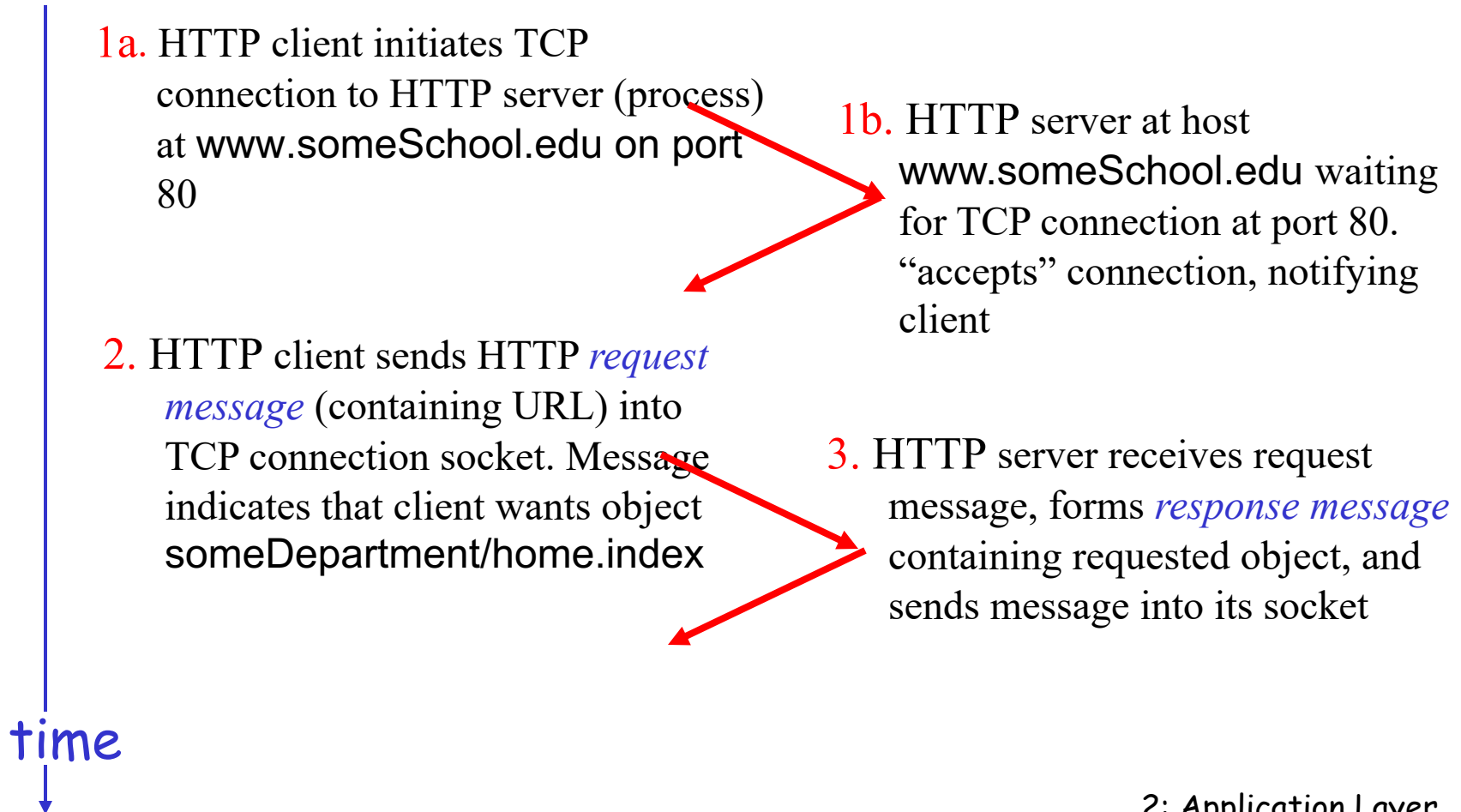
- ❑ Multiple objects can be sent over single TCP connection between client and server.
- ❑ HTTP/1.1 uses persistent connections in default mode

Nonpersistent HTTP

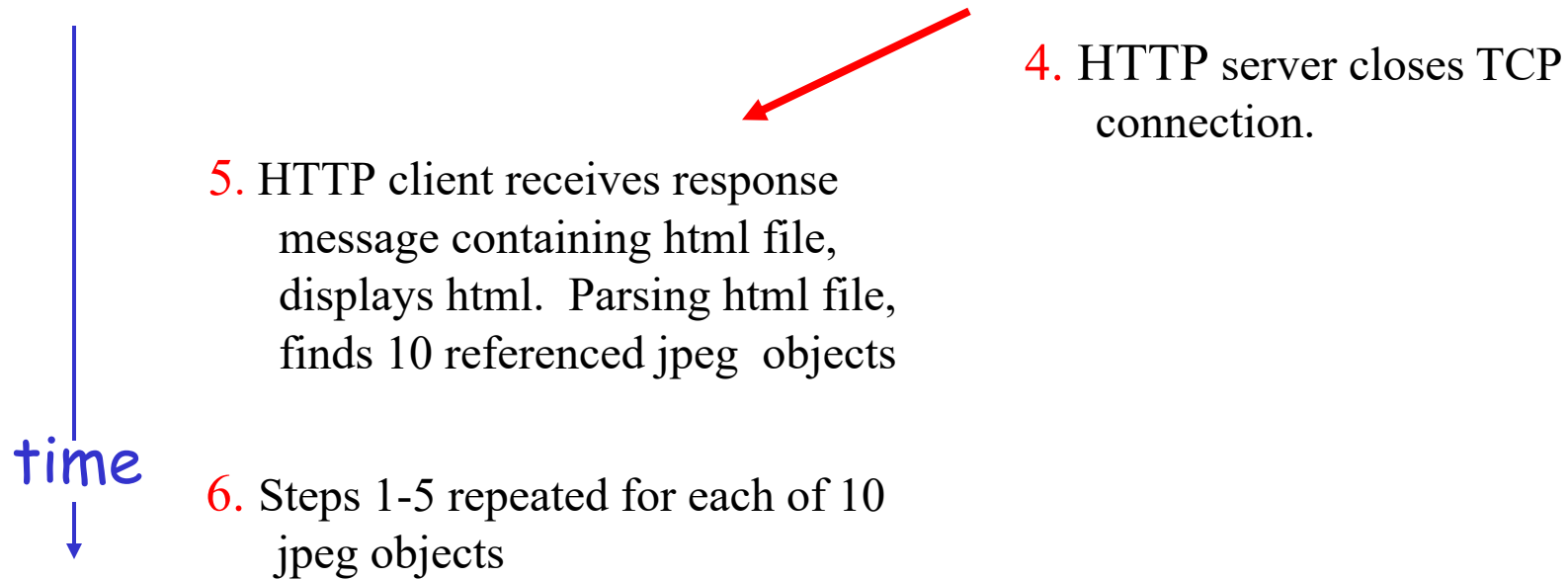
Suppose user enters URL

`www.someSchool.edu/someDepartment/home.index`

(contains text,
references to 10
jpeg images)



Nonpersistent HTTP (cont.)



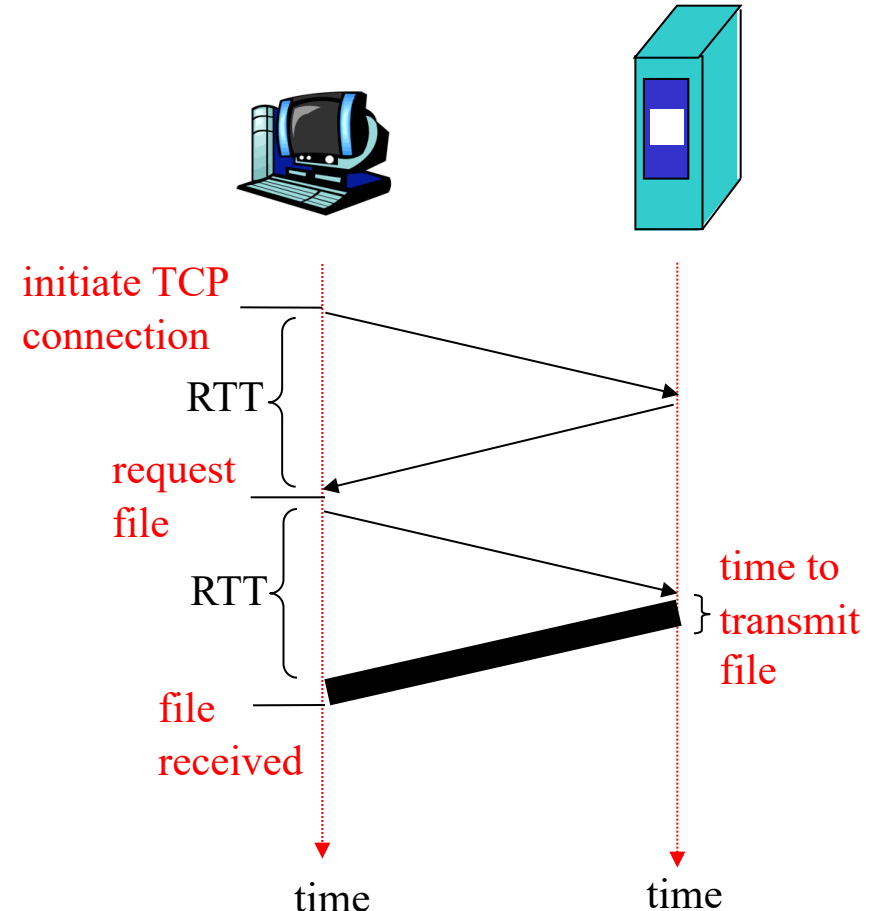
Non-Persistent HTTP: Response time

Round Trip Time (RTT) = time to send a small packet to travel from client to server and back.

Response time:

- ❑ one RTT to initiate TCP connection
- ❑ one RTT for HTTP request and first few bytes of HTTP response to return
- ❑ file transmission time

total = $2RTT + \text{<file transmit time>}$



Persistent HTTP

Nonpersistent HTTP issues:

- ❑ requires 2 RTTs per object
- ❑ OS overhead for *each* TCP connection
- ❑ browsers often open parallel TCP connections to fetch referenced objects

Persistent HTTP

- ❑ server leaves connection open after sending response
- ❑ subsequent HTTP messages between same client/server sent over open connection

Persistent *without* pipelining:

- ❑ client issues new request only when previous response has been received
- ❑ one RTT for each referenced object

Persistent *with* pipelining:

- ❑ default in HTTP/1.1
- ❑ client sends requests as soon as it encounters a referenced object
- ❑ as little as one RTT for all the referenced objects

HTTP request message

- two types of HTTP messages: *request, response*
- **HTTP request message:**
 - ❖ ASCII (human-readable format)

request line
(GET, POST,
HEAD commands)

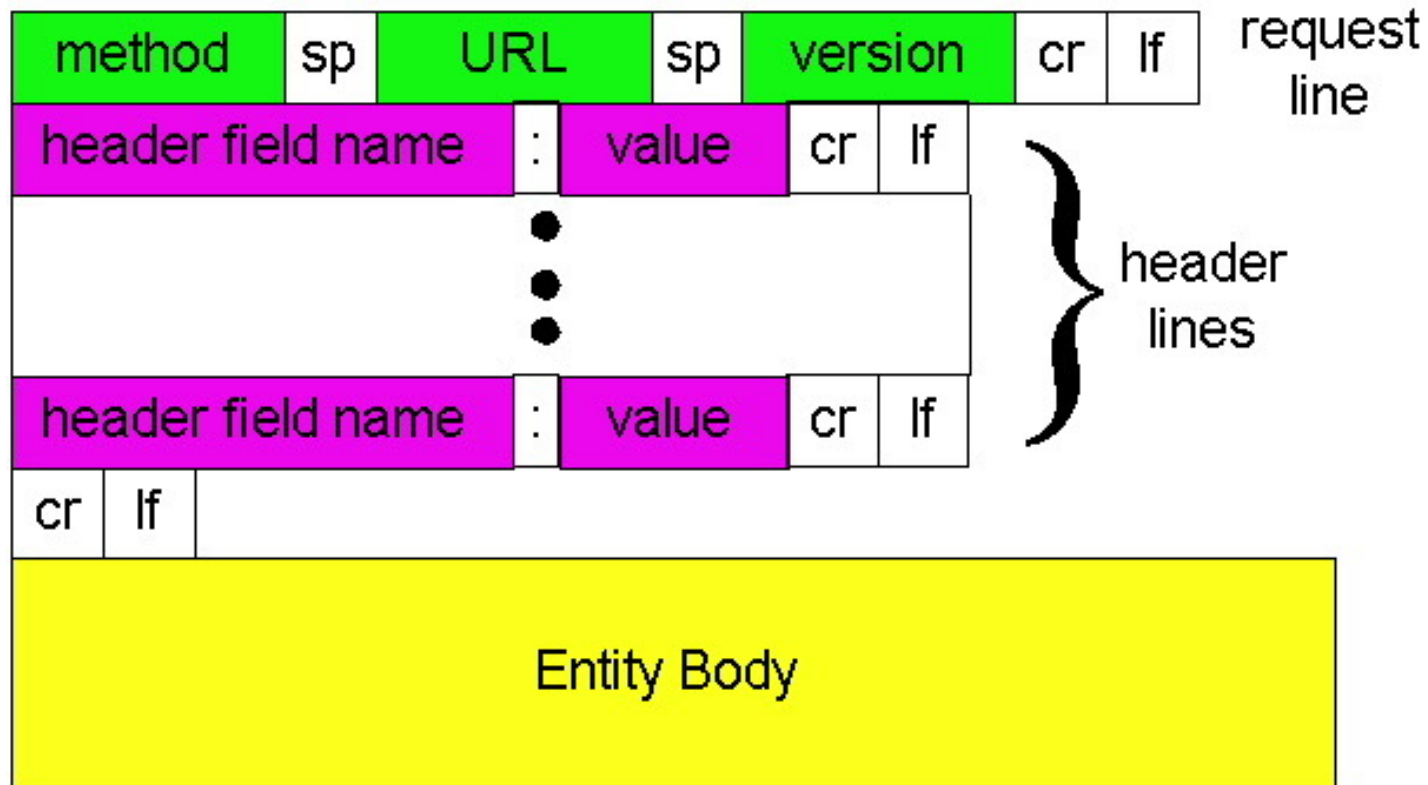
header
lines

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language: fr
```

Carriage return,
line feed
indicates end
of message

(extra carriage return, line feed)

HTTP request message: general format



Method types

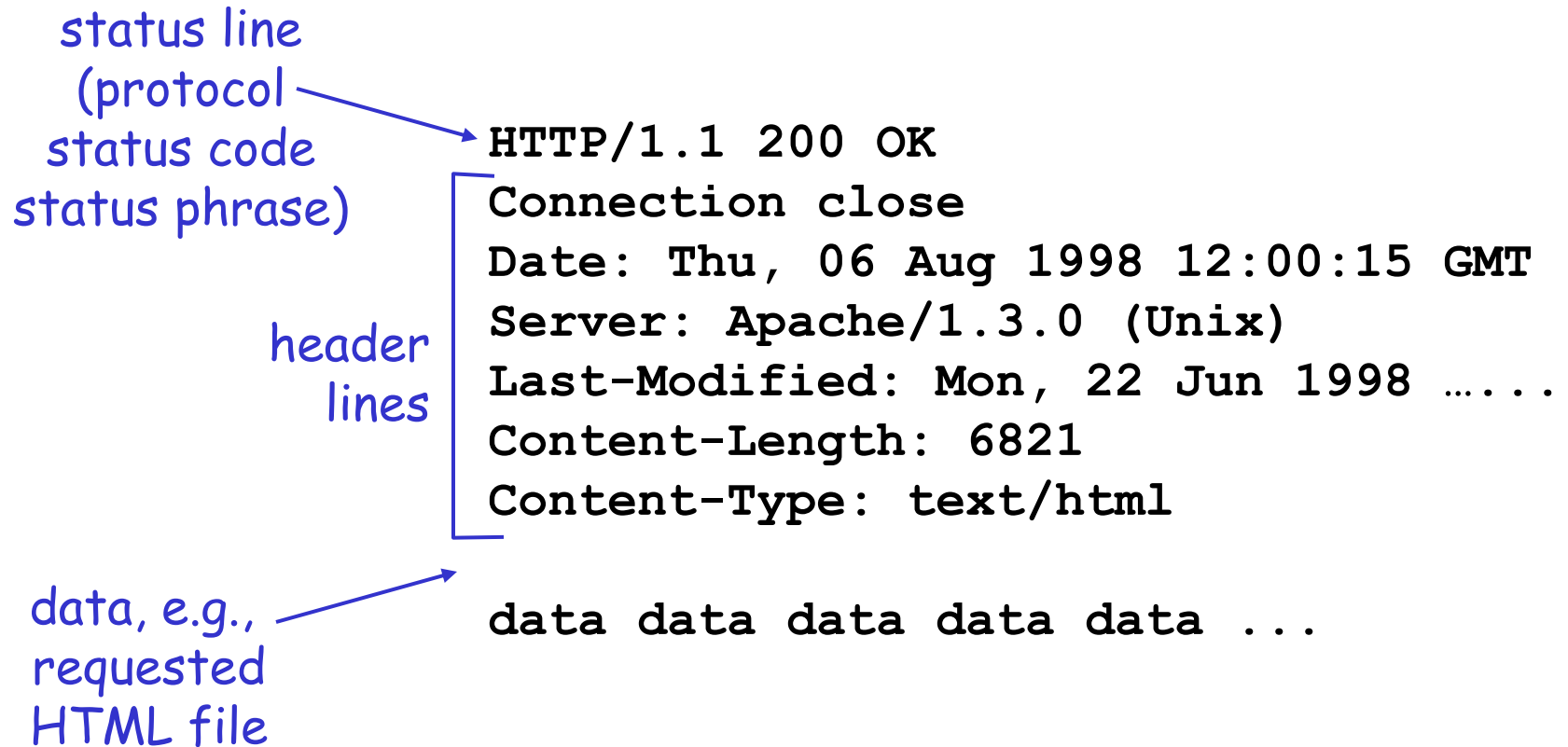
HTTP/1.0

- ❑ GET
- ❑ POST
- ❑ HEAD
 - ❖ asks server to leave requested object out of response

HTTP/1.1

- ❑ GET, POST, HEAD
- ❑ PUT
 - ❖ uploads file in entity body to path specified in URL field
- ❑ DELETE
 - ❖ deletes file specified in the URL field

HTTP response message



HTTP response status codes

In first line in server->client response message.

A few sample codes:

200 OK

- ❖ request succeeded, requested object later in this message

301 Moved Permanently

- ❖ requested object moved, new location specified later in this message
(Location:)

400 Bad Request

- ❖ request message not understood by server

404 Not Found

- ❖ requested document not found on this server

505 HTTP Version Not Supported

Chapter 2: Application layer

□ 2.1 Principles of network applications

- ❖ app architectures
- ❖ app requirements

□ 2.2 Web and HTTP

□ 2.4 Electronic Mail

- ❖ SMTP, POP3, IMAP

□ 2.5 DNS

User-server state: cookies

Many major Web sites use cookies

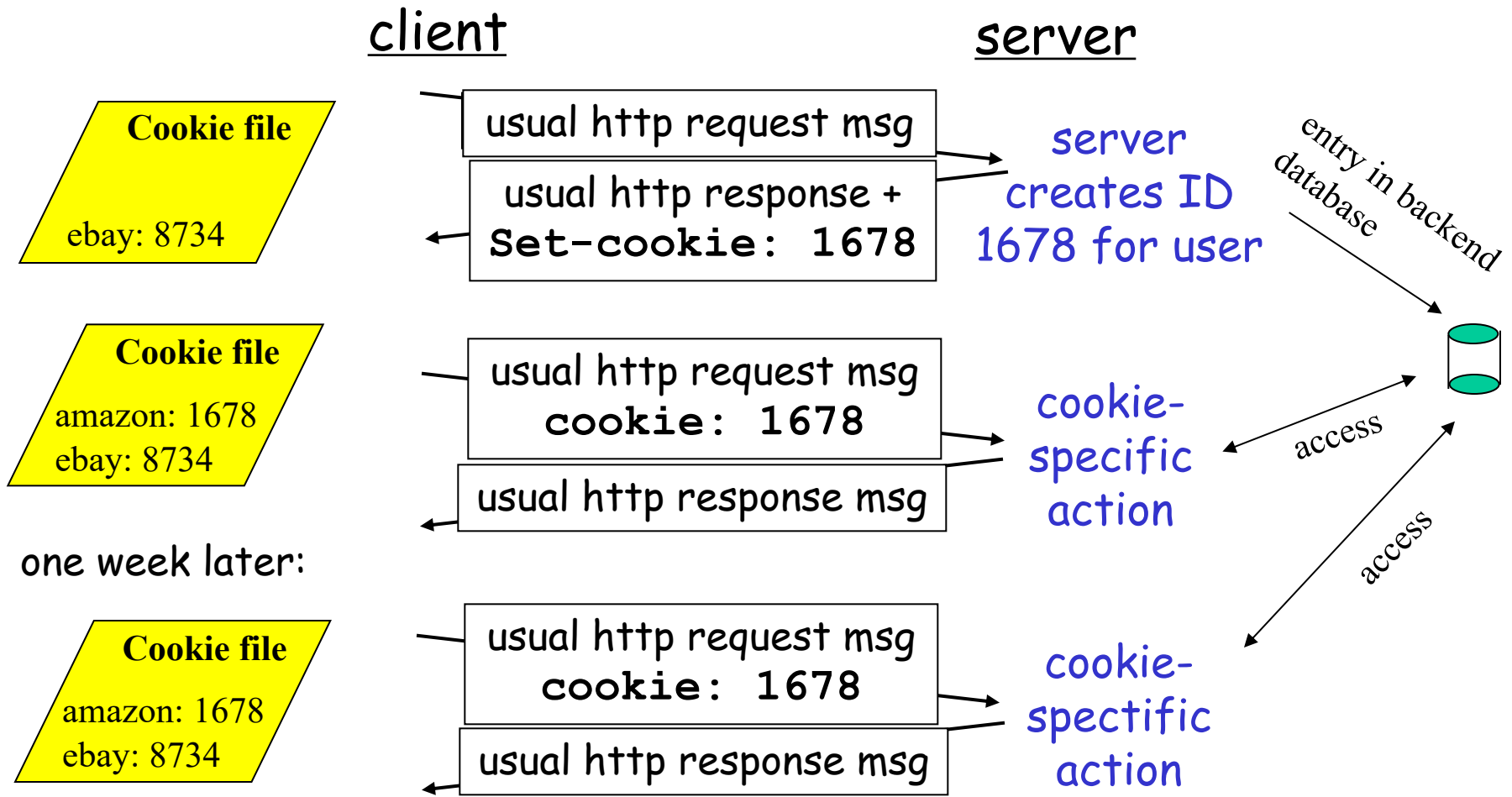
Four components:

- 1) cookie header line of HTTP *response* message
- 2) cookie header line in HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site

Example:

- ❖ Susan access Internet always from same PC
- ❖ She visits a specific e-commerce site for first time
- ❖ When initial HTTP requests arrives at site, site creates a unique ID and creates an entry in backend database for ID

Cookies: keeping “state” (cont.)



Cookies (continued)

What cookies can bring:

- ☐ authorization
- ☐ shopping carts
- ☐ recommendations
- ☐ user session state (Web e-mail)

aside

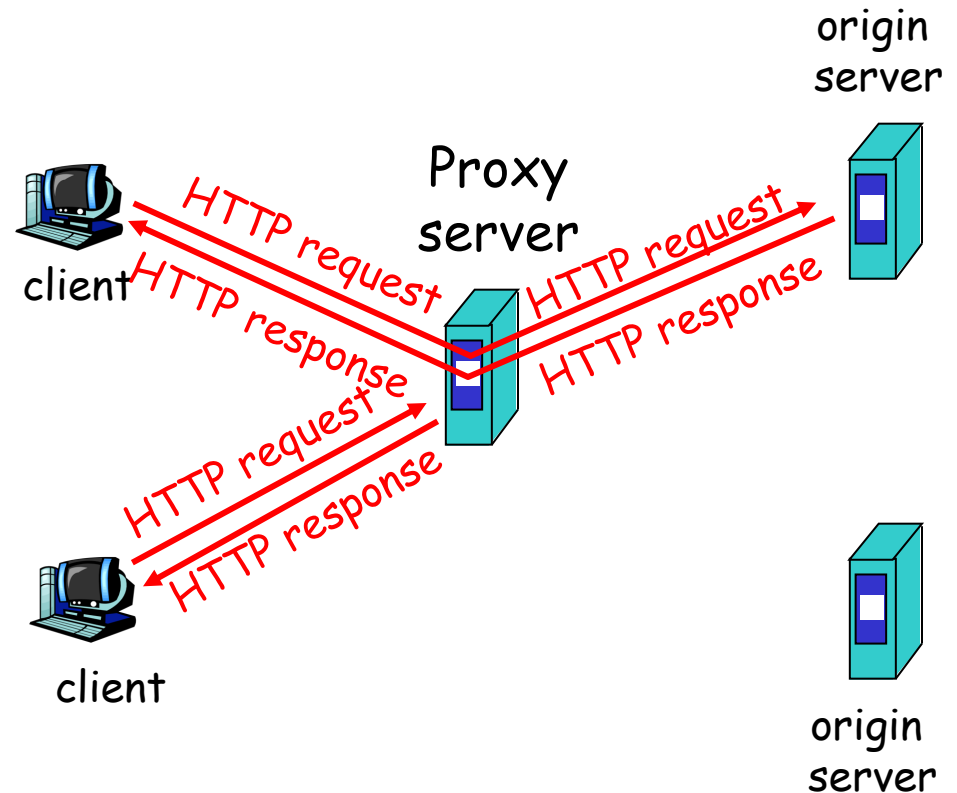
Cookies and privacy:

- ☐ cookies permit sites to learn a lot about you
- ☐ you may supply name and e-mail to sites
- ☐ search engines use redirection & cookies to learn yet more
- ☐ advertising companies obtain info across sites

Web caches (proxy server)

Goal: satisfy client request without involving origin server

- user sets browser: Web accesses via cache
- browser sends all HTTP requests to cache
 - ❖ object in cache: cache returns object
 - ❖ else cache requests object from origin server, then returns object to client



More about Web caching

- ❑ Cache acts as both client and server
- ❑ Typically cache is installed by ISP (university, company, residential ISP)

Why Web caching?

- ❑ Reduce response time for client request.
- ❑ Reduce traffic on an institution's access link.
- ❑ Internet dense with caches enables “poor” content providers to effectively deliver content (but so does P2P file sharing)

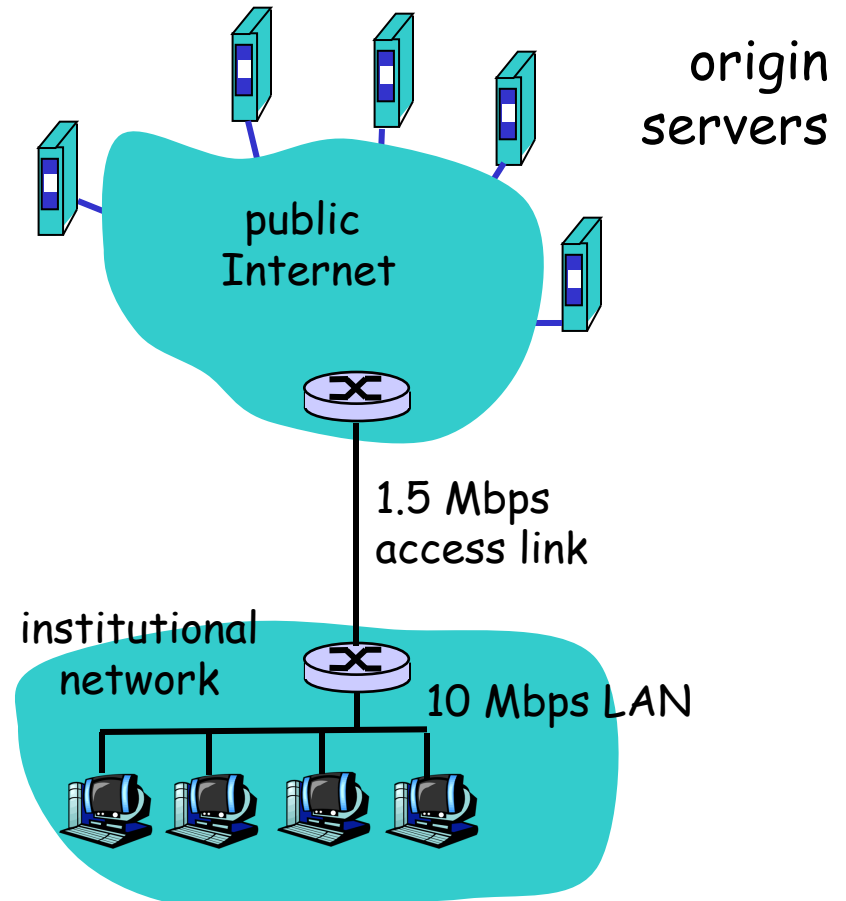
Caching example

Assumptions

- ❑ average object size = 100,000 bits
- ❑ avg. request rate from institution's browsers to origin servers = 15/sec
- ❑ delay from institutional router to any origin server and back to router = 2 sec

Consequences

- ❑ utilization on LAN = 15%
- ❑ utilization on access link = 100%
- ❑ total delay = Internet delay + access delay + LAN delay
= 2 sec + minutes + milliseconds



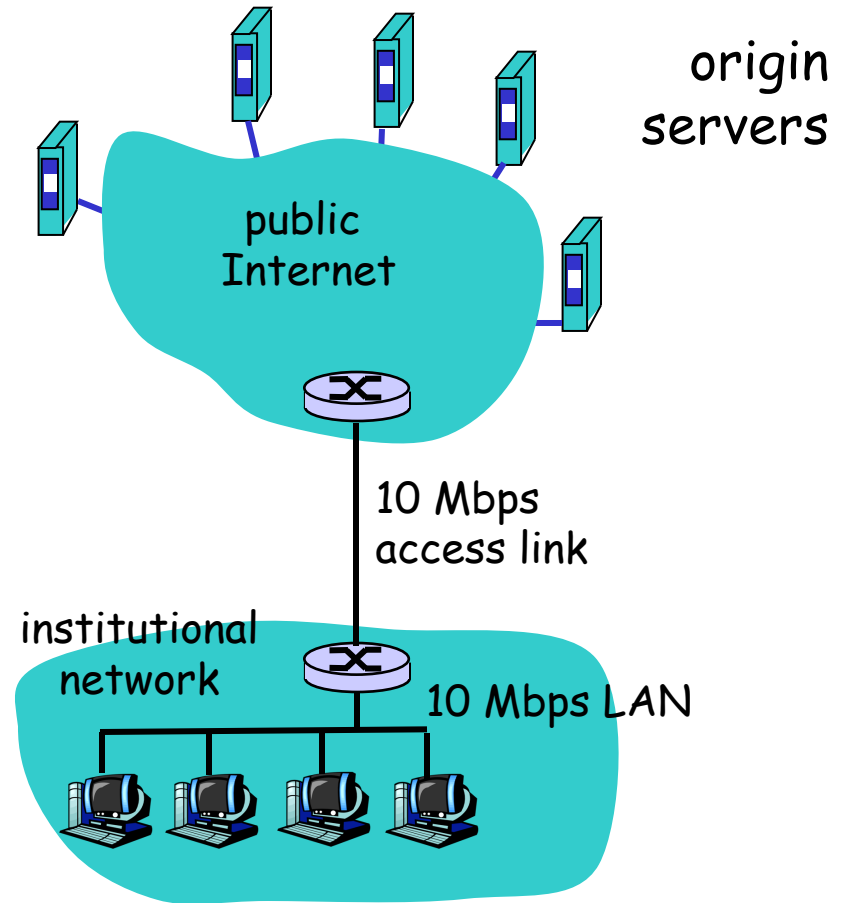
Caching example (cont)

Possible solution

- ❑ increase bandwidth of access link to, say, 10 Mbps

Consequences

- ❑ utilization on LAN = 15%
- ❑ utilization on access link = 15%
- ❑ Total delay = Internet delay + access delay + LAN delay
= 2 sec + msecs + msecs
- ❑ often a costly upgrade



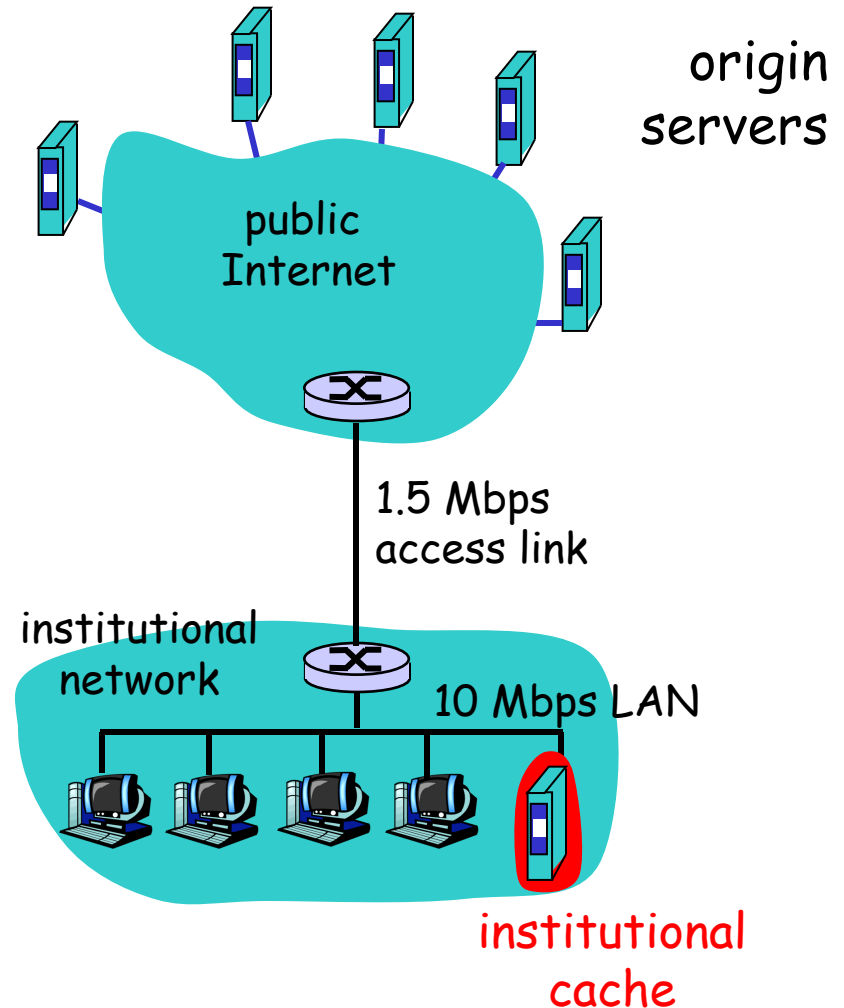
Caching example (cont)

Install cache

- suppose hit rate is .4

Consequence

- 40% requests will be satisfied almost immediately
- 60% requests satisfied by origin server
- utilization of access link reduced to 60%, resulting in negligible delays (say 10 msec)
- total avg delay = Internet delay + access delay + LAN delay = $.6 \cdot (2.01) \text{ secs} + .4 \cdot \text{milliseconds} < 1.4 \text{ secs}$



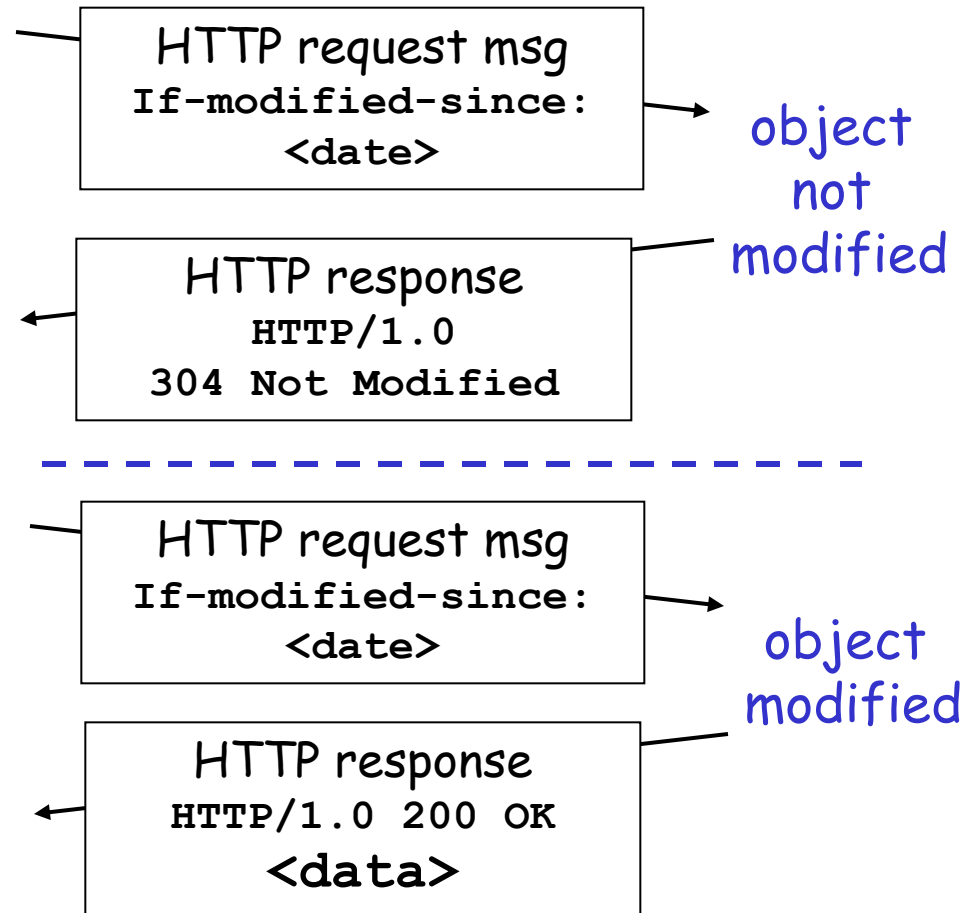
Conditional GET

- ❑ **Goal:** don't send object if cache has up-to-date cached version
- ❑ cache: specify date of cached copy in HTTP request
If-modified-since:
<date>
- ❑ server: response contains no object if cached copy is up-to-date:

HTTP/1.0 304 Not Modified

cache

server

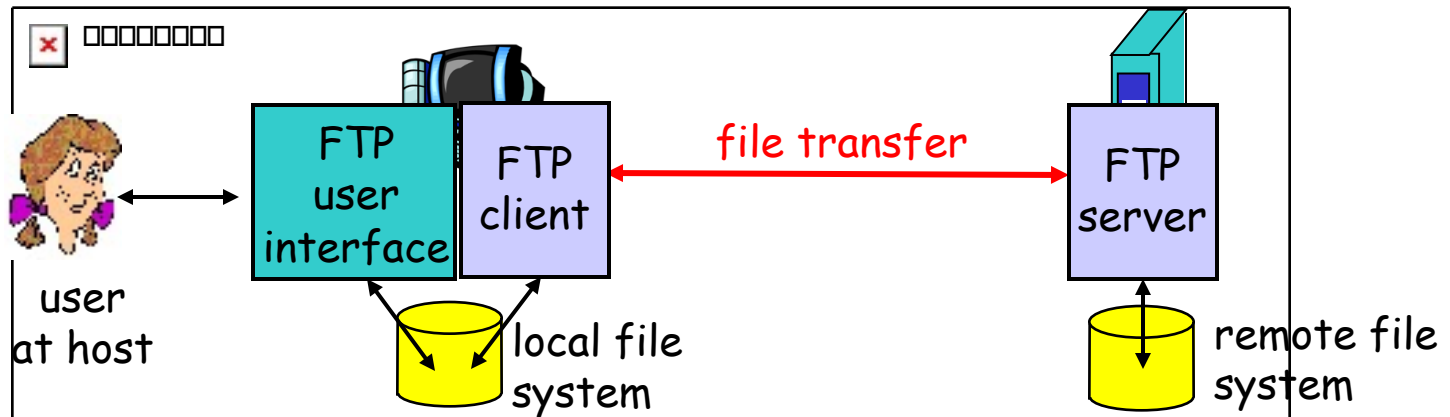


Questions?

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 P2P file sharing
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP
- ❑ 2.9 Building a Web server

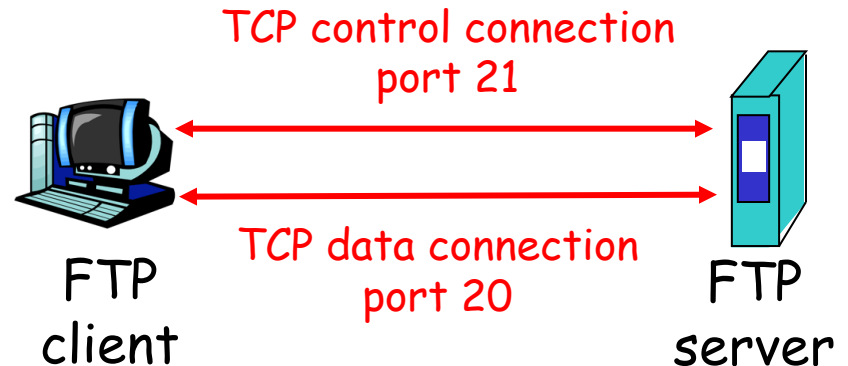
FTP: the file transfer protocol



- ❑ transfer file to/from remote host
- ❑ client/server model
 - ❖ *client*: side that initiates transfer (either to/from remote)
 - ❖ *server*: remote host
- ❑ ftp: RFC 959
- ❑ ftp server: port 21

FTP: separate control, data connections

- ❑ FTP client contacts FTP server at port 21, specifying TCP as transport protocol
 - ❖ Client obtains authorization
- ❑ Client browses remote directory by sending control commands
- ❑ When server receives a command, opens TCP data connection to client
- ❑ After transferring one file, server closes connection.



- ❑ Server opens a second TCP data connection to transfer another file.
- ❑ Control connection: “out of band”
- ❑ FTP server maintains “state”: current directory, earlier authentication

FTP commands, responses

Sample commands:

- ❑ sent as ASCII text over control channel
- ❑ **USER *username***
- ❑ **PASS *password***
- ❑ **LIST** return list of file in current directory
- ❑ **RETR *filename*** retrieves (gets) file
- ❑ **STOR *filename*** stores (puts) file onto remote host

Sample return codes

- ❑ status code and phrase (as in HTTP)
- ❑ **331 Username OK, password required**
- ❑ **125 data connection already open; transfer starting**
- ❑ **425 Can't open data connection**
- ❑ **452 Error writing file**

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS

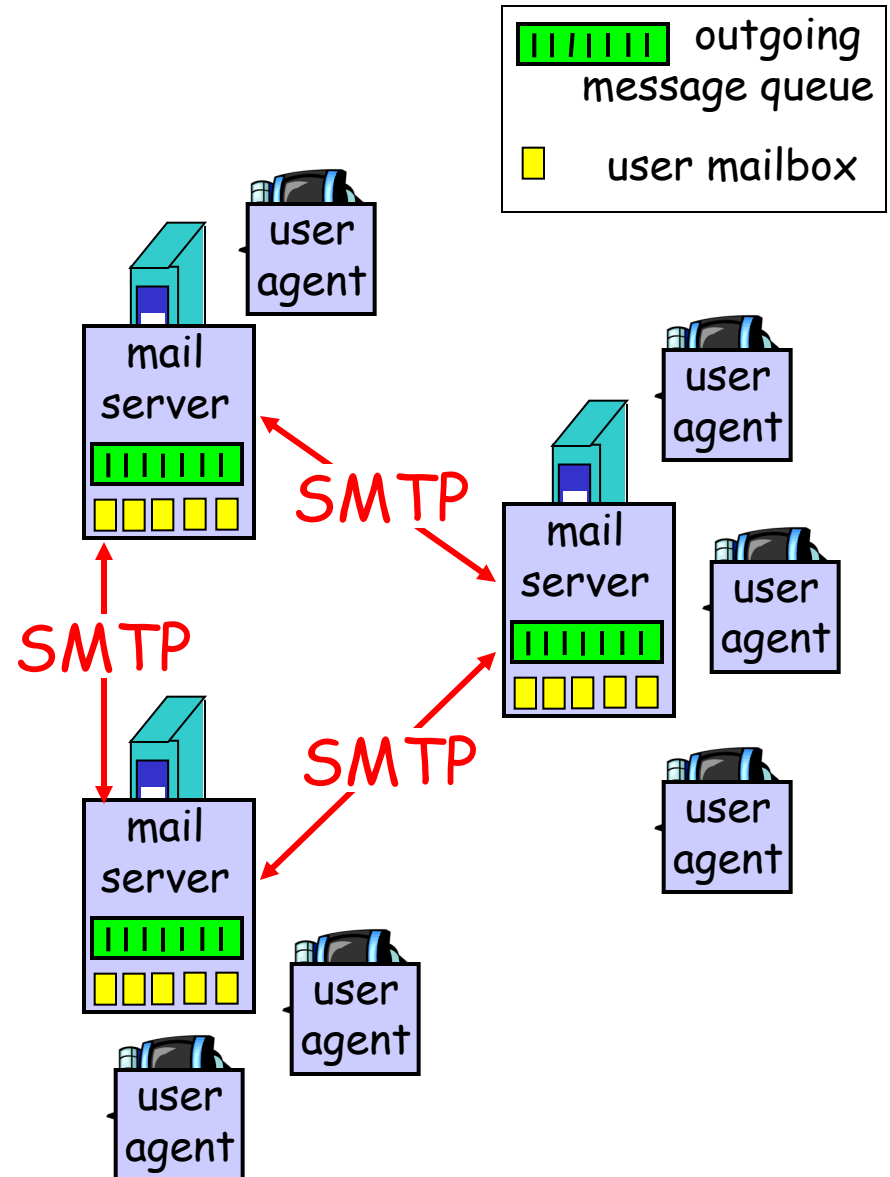
Electronic Mail

Three major components:

- ❑ user agents
- ❑ mail servers
- ❑ simple mail transfer protocol: SMTP

User Agent

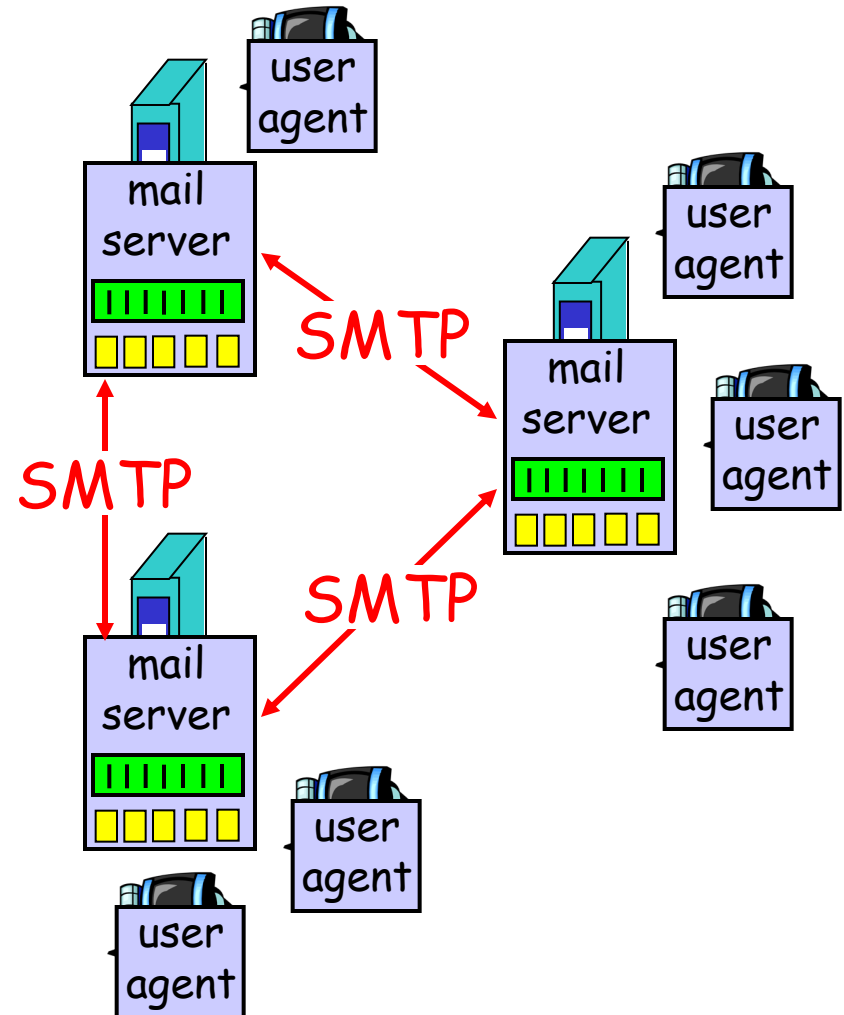
- ❑ a.k.a. “mail reader”
- ❑ composing, editing, reading mail messages
- ❑ e.g., Eudora, Outlook, elm, Netscape Messenger
- ❑ outgoing, incoming messages stored on server



Electronic Mail: mail servers

Mail Servers

- ❑ **mailbox** contains incoming messages for user
- ❑ **message queue** of outgoing (to be sent) mail messages
- ❑ **SMTP protocol** between mail servers to send email messages
 - ❖ client: sending mail server
 - ❖ “server”: receiving mail server

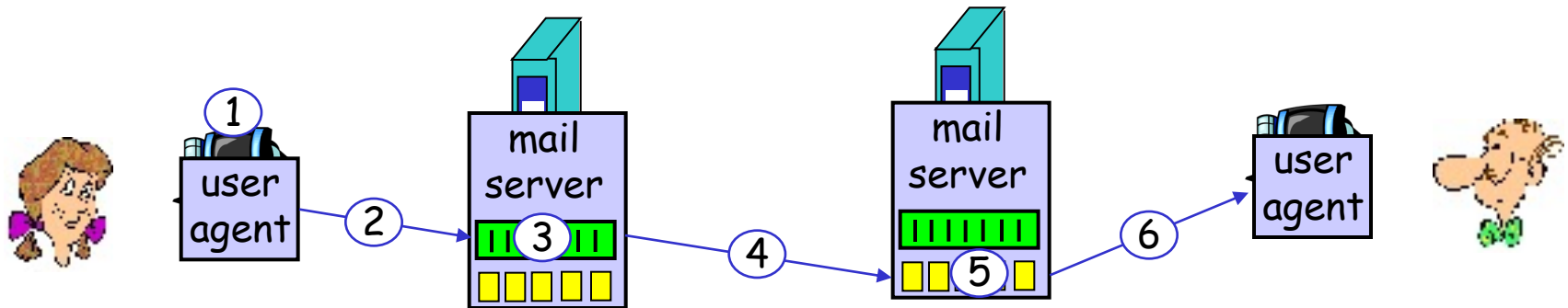


Electronic Mail: SMTP [RFC 2821]

- ❑ uses TCP on port 25 to reliably transfer email
- ❑ direct transfer: sending server to receiving server
- ❑ three phases of transfer
 - ❖ handshaking (greeting)
 - ❖ transfer of messages
 - ❖ Closure
- ❑ command/response interaction
 - ❖ **commands**: ASCII text
 - ❖ **response**: status code and phrase

Scenario: Alice Emails Bob

- 1) Alice uses UA to compose message and “to” bob@school.edu
- 2) Alice’s UA sends message to her mail server; message placed in message queue
- 3) Client side of SMTP opens TCP connection with Bob’s mail server
- 4) SMTP client sends Alice’s message over the TCP connection
- 5) Bob’s mail server places the message in Bob’s mailbox
- 6) Bob invokes his user agent to read message



SMTP Commands to send email

- ❑ Telenet into port 25
- ❑ HELO hostname
- ❑ MAIL FROM:
- ❑ RCPT TO
- ❑ RCPT TO ...
- ❑ DATA
- ❑ ... text ...
- ❑ .
- ❑ QUIT

Try SMTP interaction for yourself:

- ❑ `telnet servername 25`
- ❑ see 220 reply from server
- ❑ enter HELO, MAIL FROM, RCPT TO, DATA, QUIT commands

above lets you send email without using email client (reader)

SMTP: final words

- ❑ SMTP uses persistent connections
- ❑ SMTP requires message (header & body) to be in 7-bit ASCII
- ❑ SMTP server uses CRLF.CRLF to determine end of message

Comparison with HTTP:

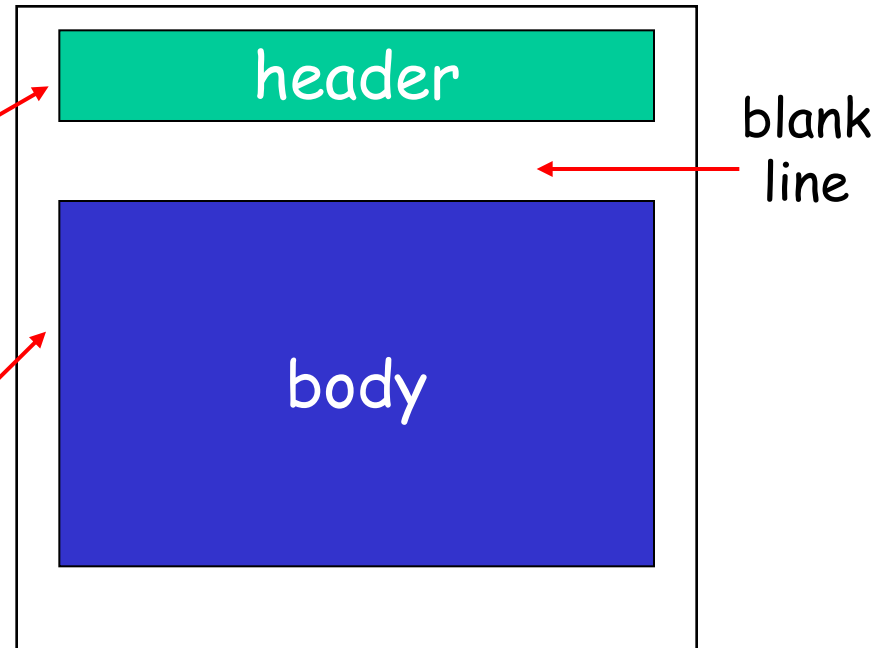
- ❑ HTTP: pull
- ❑ SMTP: push
- ❑ both have ASCII command/response interaction, status codes
- ❑ HTTP: each object encapsulated in its own response msg
- ❑ SMTP: multiple objects sent in multipart msg

Mail message format

SMTP: protocol for exchanging email msgs

RFC 822: standard for text message format:

- ❑ header lines, e.g.,
 - ❖ To:
 - ❖ From:
 - ❖ Subject:*different from SMTP commands!*
- ❑ body
 - ❖ the “message”, ASCII characters only



Message format: multimedia extensions

- ❑ MIME: multimedia mail extension, RFC 2045, 2056
- ❑ additional lines in msg header declare MIME content type
 - ❖ Think of image attachments with your email

MIME version

method used
to encode data

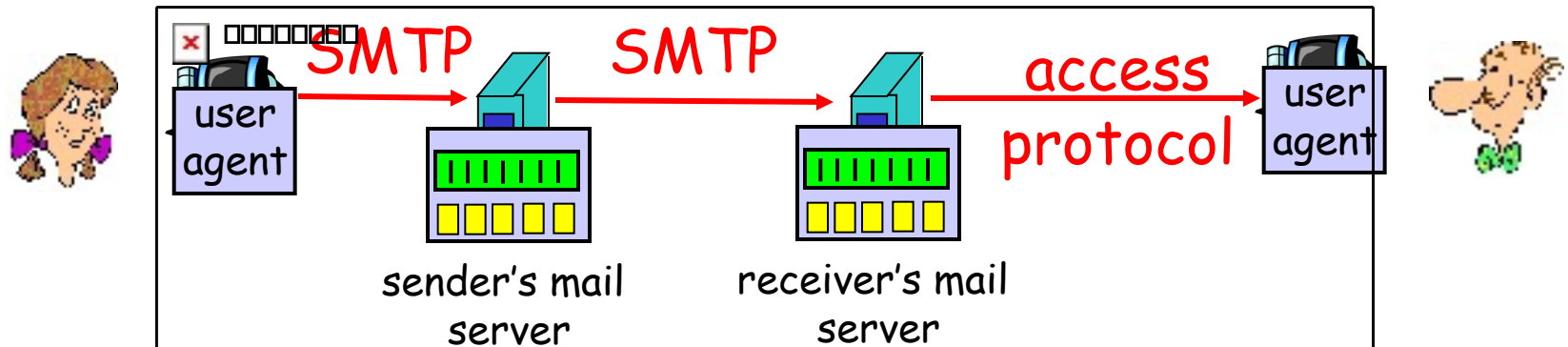
multimedia data
type, subtype,
parameter declaration

encoded data

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
.....
.....base64 encoded data
```

Mail access protocols



- ❑ SMTP: delivery/storage to receiver's server
- ❑ Mail access protocol: retrieval from server
 - ❖ POP: Post Office Protocol [RFC 1939]
 - authorization (agent <-->server) and download
 - ❖ IMAP: Internet Mail Access Protocol [RFC 1730]
 - more features (more complex)
 - manipulation of stored msgs on server
 - ❖ HTTP: Hotmail , Yahoo! Mail, etc.

What's the Difference?

POP3 protocol

authorization phase

- ❑ client commands:
 - ❖ **user**: declare username
 - ❖ **pass**: password
- ❑ server responses
 - ❖ **+OK**
 - ❖ **-ERR**

transaction phase, client:

- ❑ **list**: list message numbers
- ❑ **retr**: retrieve message by number
- ❑ **dele**: delete
- ❑ **quit**

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on

C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

POP3 (more) and IMAP

More about POP3

- ❑ Previous example uses “download and delete” mode.
- ❑ Bob cannot re-read e-mail if he changes client
- ❑ “Download-and-keep”: copies of messages on different clients
- ❑ POP3 is stateless across sessions

IMAP

- ❑ Keep all messages in one place: the server
- ❑ Allows user to organize messages in folders
- ❑ IMAP keeps user state across sessions:
 - ❖ names of folders and mappings between message IDs and folder name